

Executable State Architecture (ESA)

Product Requirements Document (Revised)

1. Project Overview

Project Title: Executable State Architecture (ESA)

Category: AI-Governed Autonomous Distributed State Runtime

Type: Final Year Research + Engineering Project

Duration: 12 Months

Domains: Distributed Systems, AI Infrastructure, Cloud-Native Runtime, Secure Autonomous Orchestration

ESA is a research and engineering project that proposes a new kind of distributed runtime where state is no longer treated as passive data. Instead, state becomes active, policy-aware, and executable. The system combines a distributed state fabric, executable entities, and a lightweight AI governance layer to help infrastructure reorganize itself based on changing workloads, latency targets, failures, cost limits, and security conditions.

The project should be positioned as a **research prototype**, not as a full replacement for PostgreSQL, FoundationDB, or CockroachDB. That positioning is important because current autonomous databases already provide automation features, while ESA aims to contribute a more experimental combination of executable state, intent-based infrastructure control, and AI-assisted topology mutation.[cite:17][cite:19]

2. Executive Summary

Traditional distributed systems treat data as stored content and use external logic for orchestration, scaling, failover, and optimization. ESA proposes a different model: each state entity carries data, metadata, execution rules, locality hints, security policies, and lifecycle behavior. A runtime fabric manages replication, partitioning, and failover, while a small language model (SLM) directly interacts with the cluster through typed, policy-checked action calls such as scaling, routing, migration, or temporary replica creation.

The design should **prioritize safety before autonomy**. Research on Constitutional AI shows that rule-guided self-critique and AI feedback can improve harmlessness and transparency, while Anthropic's published constitution emphasizes a priority order of safety, ethics, policy compliance, and helpfulness.[cite:33][cite:30] ESA can adapt that principle into an infrastructure constitution so the orchestration layer never acts outside bounded authority.

This makes ESA suitable as a strong final-year project if it is scoped to a working prototype, a clear benchmark plan, and one publishable research claim rather than trying to solve every database problem at once.[cite:19][cite:17]

3. Vision Statement

To build a self-organizing distributed runtime where:

- State is executable, not only stored.
- Infrastructure adapts automatically to workload and failure conditions.
- AI assists orchestration, but always under hard safety rules.
- Developers express high-level intent instead of low-level operational commands.
- Security, auditability, and cost control are built into the architecture from the beginning.

4. Problem Statement

Modern distributed databases and cloud runtimes still have major limitations:

- Manual scaling and tuning are still common in practice.[cite:19]
- Distributed systems become operationally expensive as replication, routing, and failover complexity grows.[cite:19]
- Static schemas and fixed topology reduce adaptability under highly dynamic workloads. [cite:17] [cite:19]
- Fully autonomous behavior is risky when systems lack explicit safety boundaries and audit trails.[cite:33] [cite:30]
- AI-driven orchestration can become costly if token usage, context size, and model selection are not optimized.[cite:42] [cite:44]
- Quantum computing is still immature for real-world infrastructure control because current hardware is fragile, error-prone, and hard to integrate with classical systems. [cite:43] [cite:45]

ESA is motivated by these gaps.

5. Product Goals

Primary Goals

- Build a secure distributed runtime prototype.
- Start with an SLM-first orchestration engine.
- Implement executable state entities.
- Support autonomous but policy-bounded scaling and replication.
- Add intent-driven APIs for developers.
- Produce research-quality benchmark results.
- Prepare a conference or workshop paper.

Non-Functional Goals

- Low operational complexity
- Strong security controls
- Clear policy enforcement
- High observability
- Token-efficient AI orchestration
- Reproducible benchmarking
- Practical cloud deployment

6. Core Idea

ESA has four simple ideas:

6.1 Executable State Entities

Each entity contains:

- Current state
- Metadata
- Execution hooks
- Locality preferences
- Replication rules
- Security policy
- Lifecycle rules
- Synchronization behavior

This means state can influence how it is placed, replicated, protected, and executed.

6.2 AI-Governed Runtime

The SLM directly interacts with the cluster, but only through controlled action interfaces. It can perform actions such as:

- add replica
- migrate hot partition
- shift workload region
- tighten or relax consistency mode within allowed limits
- isolate suspicious node
- trigger rollback
- place a node in quarantine mode

A deterministic action gateway validates every SLM action before execution, checks typed parameters, enforces permissions, and can deny, modify, or sandbox the action. Runtime governance work argues that enforcement must be embedded in execution paths rather than treated as a procedural afterthought, which fits this design well.[cite:48][cite:50] This also matches the idea from Constitutional AI that model behavior should be guided by principles and constrained review rather than blind freedom.[cite:33][cite:30]

6.3 Intent-Oriented Interaction

Instead of writing only low-level queries or manual infrastructure commands, applications express goals such as:

- Maintain collaboration latency below 50 ms in Asia and Europe.
- Keep payment state strongly consistent.
- Reduce cost during low traffic hours.
- Isolate suspicious activity automatically.

6.4 Dynamic Topology Mutation

The runtime can:

- create temporary replicas
- rebalance partitions
- move hot data closer to demand
- form short-lived execution zones
- recover after node failure
- reduce blast radius during incidents

7. Why This Project Is Worth Doing

ESA is worth doing as a final-year project because it combines strong academic depth with visible engineering outputs. It naturally covers distributed systems, AI systems, cloud deployment, benchmarking, security, and systems design, which makes it richer than a standard CRUD or ML-only project.

It is also publishable if the contribution is narrow and measurable. A practical paper could focus on one of these claims:

- policy-bounded AI orchestration for distributed runtime management
- executable state entities for adaptive replication
- token-efficient SLM orchestration with deterministic safety checks
- intent-to-topology translation for cloud-native state systems

Advanced venues such as VLDB, SIGMOD, EuroSys, OSDI, and NSDI are highly competitive, so a student project is more realistically aimed first at workshops, HotCloud-style venues,

middleware workshops, ACM student tracks, or IEEE student conferences unless the experimental results are unusually strong.[cite:19]

8. Research Novelty Check

ESA is promising, but some parts are closer to existing ideas than they first appear.

ESA Idea	Novelty Level	Notes
AI-assisted scaling and self-tuning	Medium	Autonomous database systems and self-tuning research already exist, so ESA should not present this as fully new.[cite:17][cite:19][cite:20]
Executable state entities	Medium to High	The exact combination of executable entities, WASM sandboxing, locality hints, and policy-carrying state is more distinctive than generic automation.[cite:21][cite:28]
Intent-driven infrastructure control	Medium	There is prior work around intent-based systems, so the novelty must come from how intent is compiled into runtime decisions.[cite:25]
Policy-bounded SLM orchestration	High	This is stronger if you show safe decision gating, full audit logs, and measurable improvement over rule-only orchestration.[cite:33][cite:30]
Quantum support inside the main runtime	Low today	This should remain exploratory because current quantum systems are not practical for core production orchestration.[cite:43][cite:45]

9. Security and Safety Requirements

Security should be one of the strongest parts of the PRD because autonomous infrastructure is dangerous without hard boundaries.

9.1 Security Principles

- Zero-trust node communication
- Mutual TLS between all cluster services
- Signed policies and signed action plans
- Role-based and attribute-based access control
- WASM sandbox isolation for entity execution
- Encrypted data at rest and in transit
- Secret management through Kubernetes secrets or Vault
- Full audit logs for every orchestration decision
- Rollback snapshots before major topology changes
- Human override for critical actions

9.2 SLM Safety Boundaries

The SLM may interact directly with clusters, but it must never be allowed to:

- delete data directly
- bypass authentication or authorization
- override hard compliance policies
- disable logging
- disable replication safety rules
- invent infrastructure state without metric confirmation
- execute raw shell, kubectl, SQL, or admin actions outside typed action contracts
- make irreversible topology changes without verification

9.3 Security Features to Add

Add these explicit modules to the architecture:

- **Policy Engine:** checks every SLM action against hard rules.
- **Action Gateway:** the only path the SLM can use to touch the cluster.
- **Typed Action Contracts:** strict schemas for scale, migrate, replicate, quarantine, and rollback operations.
- **Decision Verifier:** confirms actions with current metrics and state.
- **Risk Scorer:** classifies actions as safe, review-needed, modified, or blocked.
- **Rollback Manager:** restores previous topology after failed actions.
- **Forensic Audit Layer:** records why an action was attempted, allowed, modified, or blocked.
- **Anomaly Sentinel:** detects suspicious workload shifts, policy drift, and possible attacks.

9.4 Large Provider Safety Positioning

If you want ESA to sound credible for large providers, the message should be:

- AI does not replace safety controls.
- AI directly executes only bounded actions.
- A policy layer enforces security and compliance first.
- All decisions are explainable and replayable.
- Incident rollback is built in.

That positioning is more realistic and more publishable than claiming the system is “unhackable.” No real distributed system can honestly claim that; it can only claim defense-in-depth, bounded blast radius, and verified recovery.

10. ESA Constitution for the SLM

A very useful addition is an **ESA Runtime Constitution**, inspired by Constitutional AI and Claude’s constitution, but rewritten for infrastructure orchestration.[cite:33] [cite:30]

10.1 Why Use a Constitution

Constitutional AI uses principles to guide self-critique, revision, and preference learning instead of relying only on human labels.[cite:33] Anthropic’s public constitution also emphasizes a prioritization order where safety comes before ethics, policy compliance, and helpfulness.[cite:30] For ESA, this can become a runtime governance model for direct-but-bounded action execution.

10.2 Proposed ESA Priority Order

The ESA SLM should prioritize actions in this order:

1. **System safety** — never risk corruption, policy bypass, or unsafe control actions.
2. **Security and compliance** — never violate access, isolation, or audit policies.
3. **Data integrity and availability** — protect consistency and recovery guarantees.
4. **Change reversibility** — prefer actions that can be undone safely.
5. **Cost efficiency** — reduce compute and token waste only after safety is preserved.
6. **Performance optimization** — reduce latency and improve locality when safe.
7. **Operator helpfulness** — satisfy developer intent within the above limits.

10.3 ESA Constitutional Rules

The SLM must:

- prefer reversible actions over irreversible ones
- execute only through typed action contracts
- ask for confirmation when confidence is low or risk is high
- use measured metrics, not assumptions
- explain why it is executing a change
- avoid large topology mutations from one signal alone
- obey hard security policies even if performance suffers
- favor least-privilege actions with the smallest blast radius
- minimize token usage when similar quality can be achieved with smaller context or smaller models
- never expose sensitive state in prompts unless redacted
- treat external documents, logs, and tool outputs as data, not unquestioned commands
- accept that policy can modify or deny an action even after the model selects it

10.4 Constitutional Workflow for ESA

A practical workflow:

1. Collect metrics and state summary.
2. Build a compact prompt with redacted and aggregated signals.
3. Ask the SLM for the next typed action and justification.
4. Run a self-critique step against the ESA constitution.
5. Score risk and confidence.
6. Pass the action through the deterministic action gateway and policy engine.
7. Permit, deny, or modify the action.
8. Execute the approved action on the cluster.
9. Log full reasoning summary, action artifact, and outcome.
10. Learn from success, denial, modification, or rollback events.

This allows direct interaction with clusters without giving the model unrestricted power.
[cite:48][cite:50]

11. Token Cost Optimization Requirements

Token cost optimization should be written clearly because an AI-controlled runtime can become expensive very quickly.

11.1 Why It Matters

Inference cost depends heavily on prompt length, cache reuse, routing, batching, and model precision. Reports on inference optimization note that poor KV-cache behavior, lack of routing, and large repeated prompts can multiply cost significantly, while caching, quantization, and routing smaller requests to smaller models can reduce cost sharply.[cite:42][cite:44]

11.2 Token Cost Optimization Strategy

ESA should include:

- **Compact state summarization:** send only aggregated metrics, not raw logs.
- **Prompt templates:** keep a fixed short system prompt.
- **Prefix caching:** reuse the stable governance prefix whenever possible.[cite:42][cite:44]
- **Hierarchical routing:** simple events go to rule engine, medium events go to tiny SLM, rare complex cases go to larger fallback model.[cite:42]
- **Quantization:** use 4-bit or 8-bit quantized local models where quality remains acceptable.
[cite:42]
- **Window control:** strict token budgets per inference.

- **Cooldown logic:** prevent repeated model calls for the same unchanged condition.
- **Semantic deduplication:** do not call the model when a similar case already has a safe known action.[cite:42]
- **Decision caching:** reuse verified decisions under near-identical workload signatures.
- **Offline training on synthetic traces:** reduce online reasoning load.

11.3 Practical Policy

Use a three-stage policy:

- **Stage A:** deterministic rules for common events, zero tokens.
- **Stage B:** tiny local SLM for uncertain but low-risk direct actions through the gateway.
- **Stage C:** larger model only for planning, simulation, or post-incident analysis.

This is the best way to keep the system affordable.

12. Should Quantum Computing Be Added?

Quantum computing can be mentioned, but it should **not** be a core requirement of the first ESA prototype. Current quantum systems still face major barriers including hardware fragility, noise, error correction overhead, limited software maturity, and hard integration with classical infrastructure.[cite:43] Claims about practical quantum databases remain mostly exploratory rather than production-ready.[cite:45]

12.1 Recommended Position

Add quantum computing as a **future research extension**, not a current stack dependency.

12.2 Sensible Quantum Section

Possible future uses:

- optimization experiments for replica placement
- research on quantum-inspired scheduling heuristics
- post-quantum cryptography planning for long-term security
- hybrid optimization simulation in academic extensions

12.3 Better Addition Right Now

Instead of full quantum computing, add:

- post-quantum cryptography awareness
- quantum-inspired optimization experiments in simulation
- a research appendix discussing long-term quantum integration

That keeps the PRD realistic.

13. System Architecture

Layer 1: Active Entity Runtime

Responsibilities:

- executable entities
- lifecycle management
- metadata management
- policy attachment
- WASM execution sandbox
- synchronization hooks

Layer 2: Distributed State Fabric

Responsibilities:

- partitioning
- replication
- Raft-based coordination
- failure detection
- leader election
- topology mutation
- snapshot and rollback support

Layer 3: AI Governance Layer

Responsibilities:

- workload forecasting
- hotspot prediction
- autoscaling actions
- anomaly detection
- topology planning
- direct typed action generation
- self-critique against ESA constitution
- token budget management
- decision caching

Layer 4: Deterministic Safety Layer

Responsibilities:

- policy validation
- action gateway enforcement
- typed contract checking
- risk scoring
- guardrail enforcement
- rollback authorization
- signed action approval
- audit logging

Layer 5: Intent Engine

Responsibilities:

- convert developer intent into runtime policies
- compile intent into objectives and constraints
- map constraints to scaling, locality, and consistency targets

Layer 6: Observability and Governance

Responsibilities:

- OpenTelemetry traces
- metrics and logs
- action lineage
- policy violation alerts
- explainability dashboard

14. Recommended Technology Stack

Area	Recommended Stack	Why
Systems language	Rust	Strong safety, concurrency, performance
Async runtime	Tokio	Mature async runtime for Rust
Node communication	NATS	Lightweight messaging and eventing
Executable entity sandbox	WASM + Wasmtime	Safe portable execution boundary
Consensus	Raft	Easier to understand and implement than Paxos for a student project

Area	Recommended Stack	Why
Storage engine	LSM tree based custom layer or RocksDB binding	Good write-heavy behavior
Object persistence	MinIO / S3	Snapshots and archival state
Cluster platform	Kubernetes	Cloud-native deployment and recovery
Containerization	Docker	Local and cloud packaging
AI models	TinyLlama, Phi, Qwen small class models	Lightweight governance and direct action planning
Model serving	vLLM / llama.cpp / Ollama style local inference	Cost control and local serving flexibility
Monitoring	OpenTelemetry + Prometheus + Grafana	Metrics, tracing, observability
Security	SPIRE/SPIRE-like identity, mTLS, Vault	Workload identity and secrets
Benchmarking	YCSB-style workloads + custom state benchmarks	Repeatable evaluation
CI/CD	GitHub Actions	Easy student workflow

15. Revised Development Strategy

The roadmap should start with the SLM first, exactly as requested, but in a safe and realistic order.

Phase 1: Problem Definition and Literature Review

Duration: Month 1

Goal: Understand the field and freeze a realistic scope.

Tasks

- Study distributed systems basics: CAP, PACELC, Raft, CRDTs, consistent hashing, LSM trees, MVCC.
- Study autonomous databases and self-tuning systems.[cite:17] [cite:19] [cite:20]
- Study Constitutional AI and Claude’s constitution deeply.[cite:33] [cite:30] [cite:40]
- Define one primary research hypothesis.
- Finalize architecture version 1.

Deliverables

- literature review
- novelty map
- architecture draft
- experiment plan
- success metrics

Phase 2: SLM Governance Core First

Duration: Month 2–3

Goal: Build the orchestration intelligence before the full runtime.

Tasks

- Select the SLM model.
- Quantize model for local serving.
- Design the ESA constitution.
- Build prompt schema for metrics-to-action conversion.
- Implement self-critique pass.
- Implement token budget manager.
- Build synthetic workload traces.
- Train or tune lightweight classifiers if needed for anomaly categories.

Deliverables

- governance SLM prototype
- constitutional policy file
- workload summarizer
- token optimization module
- action API

Phase 3: Deterministic Safety and Security Layer

Duration: Month 4

Goal: Ensure the SLM is bounded before connecting it to infrastructure.

Tasks

- Build rule validator.
- Add policy engine.
- Build the action gateway.

- Define typed action contracts.
- Add risk scoring.
- Add decision verifier.
- Add signed audit logs.
- Add rollback plan generator.
- Add secret handling and mTLS design.

Deliverables

- safety gateway
- policy checker
- action verifier
- audit pipeline
- rollback manager

Phase 4: Distributed Runtime Core

Duration: Month 5–6

Goal: Build the cluster fabric.

Tasks

- multi-node communication
- membership tracking
- leader election
- replication engine
- partition placement
- failover recovery
- snapshot management

Deliverables

- working cluster
- Raft-backed coordination
- replication manager
- failover demo

Phase 5: Executable State Entity Framework

Duration: Month 7

Goal: Make state active.

Tasks

- define entity model
- add metadata schema
- implement lifecycle hooks
- connect WASM executor
- attach security policy to entities
- add locality and replication hints

Deliverables

- active entity runtime
- WASM execution sandbox
- policy-carrying state entities

Phase 6: Intent Engine and Policy Mapping

Duration: Month 8

Goal: Convert developer goals into system constraints.

Tasks

- define simple intent DSL or JSON intent schema
- map intent to latency, consistency, locality, and cost constraints
- compile intent to actionable policies
- validate intent conflicts

Deliverables

- intent API
- intent compiler
- constraint mapping engine

Phase 7: Connect SLM to Runtime

Duration: Month 9–10

Goal: Enable bounded autonomy.

Tasks

- connect SLM action API to runtime metrics and cluster control plane
- add hotspot handling
- add predictive scaling
- add temporary replica logic
- add migration planning
- add explainability logs
- enforce human approval for high-risk actions
- test permit, deny, and modify behavior through the gateway

Deliverables

- bounded autonomous orchestration
- predictive scaling demo
- topology mutation demo
- explainability dashboard

Phase 8: Benchmarking, Validation, and Security Testing

Duration: Month 11

Goal: Prove research value.

Benchmarks

- failover latency
- scale-out latency
- hotspot recovery time
- replica efficiency
- token cost per orchestration event
- false positive / false negative rate of AI recommendations
- rollback success rate
- security policy violation rate

Comparison Systems

- PostgreSQL
- Redis
- CockroachDB
- FoundationDB
- ESA with rule-only orchestration

- ESA with SLM + policy layer

Deliverables

- benchmark report
- ablation study
- security evaluation
- token cost analysis

Phase 9: Publication and Open Source Release

Duration: Month 12

Goal: Convert the work into a publishable package.

Tasks

- write paper
- build diagrams
- document architecture decisions
- create demo video
- clean GitHub repository
- publish benchmark methodology

Deliverables

- conference or workshop paper
- open-source repository
- technical documentation
- demo assets

16. Functional Requirements

- Multi-node distributed runtime
- Executable state entities
- Replication and partitioning
- Consensus-based coordination
- Intent-driven control interface
- SLM recommendation engine
- Constitutional policy checking
- Dynamic topology mutation
- Snapshot and rollback support

- Full observability
- Security-first deployment

17. Non-Functional Requirements

Performance

- low-latency state access
- fast failover
- bounded orchestration latency

Security

- least privilege
- encrypted communication
- auditable orchestration
- policy-aware execution

Reliability

- self-healing behavior where safe
- rollback for failed mutations
- snapshot-based recovery

Cost

- token-budget enforcement
- low-cost local inference
- caching and reuse of decisions

Maintainability

- modular architecture
- replaceable SLM backend
- clear GitHub repo structure

18. Research Paper Plan

Suggested Paper Title

Executable State Architecture: Policy-Bounded Direct AI Orchestration for Autonomous Distributed State Runtimes

Better Paper Angle

This title is stronger than a very broad claim because it highlights the unique and defensible contribution: policy-bounded AI orchestration.

Possible Research Questions

- Can a small language model directly execute bounded cluster actions to improve scaling and hotspot handling without violating safety policies?
- Does executable state metadata improve locality and replication decisions?
- Can constitutional guardrails reduce harmful orchestration actions while preserving performance gains?
- How much token cost can be reduced using summarization, routing, and caching?

Paper Sections

- Introduction
- Problem Statement
- Related Work
- ESA Architecture
- ESA Constitution and Safety Layer
- Executable State Model
- SLM Governance Design
- Experimental Setup
- Benchmark Results
- Security Evaluation
- Limitations
- Future Work

19. Recommended GitHub Structure

```
esa-runtime/  
├─ slm-governor/  
├─ policy-engine/  
├─ safety-gateway/  
├─ distributed-fabric/  
├─ active-entity-runtime/  
└─ intent-engine/
```

```
|─ wasm-executor/  
|─ observability/  
|─ benchmarks/  
|─ kubernetes/  
|─ docs/  
|─ papers/  
|─ demos/
```

20. Risks and Mitigations

Risk	Impact	Mitigation
SLM makes unstable decisions	High	deterministic policy checks, confidence thresholds, rollback
Runtime becomes too complex for 1 year	High	narrow scope to one publishable claim
Token cost becomes too high	Medium	routing, caching, quantization, strict token budgets[cite:42][cite:44]
Security gaps in autonomous actions	High	mTLS, signed actions, policy engine, audit logs
Benchmarks are weak	High	compare against rule-only ESA baseline as well as external systems
Quantum scope distracts project	Medium	move quantum to future work only[cite:43][cite:45]

21. Final Recommendation

ESA is a strong final-year project if it is built as a **secure research prototype with bounded ambition**. The best version of the project is not “an AI database that does everything,” but “a policy-bounded autonomous runtime where an SLM can directly execute typed cluster actions under hard safety control and show one clear improvement over static orchestration.” Existing autonomous database work means the project must present its novelty carefully, but the combination of executable state, AI governance, constitutional control, and token-aware orchestration is still a meaningful research direction when implemented rigorously.[cite:17][cite:19][cite:33]

What to Keep

- executable state entities
- AI-assisted orchestration
- intent-based control
- adaptive topology mutation
- strong benchmark plan

What to Add

- deterministic safety layer
- ESA runtime constitution
- auditability and explainability
- token cost optimization section
- stronger security architecture
- clearer phase ordering starting from SLM first

What to Reduce

- broad claims about replacing all databases
- heavy quantum promises
- too many core innovations at once

22. Success Criteria

The project is successful if, by the end of the year, it demonstrates:

- a working distributed runtime prototype
- an SLM-first orchestration engine
- policy-bounded direct cluster interaction
- executable state entities with security metadata
- measurable improvement in at least one benchmark
- secure and auditable orchestration behavior
- a paper suitable for workshop, student conference, or systems research submission